CONFLUENT

# PLATFORM

Enterprise Kafka —
Features, Clients & Tools

**Rajesh Pasham**

### Enterprise Features

Control Center, Kubernetes, Cluster Linking…

### Balancing & Replication

Self-Balancing, Auto Data Balancer, Replicator

### Schema Registry

Zero-downtime schema evolution & governance

### REST Proxy & Clients

HTTP, C/C++, Python, Go, .NET client libs

### ksqlDB & Community

Streaming SQL, Connectors, Tiered Storage

# Confluent Platform vs Apache Kafka

*Every Confluent release bundles the latest Kafka plus enterprise-grade additions*

## Apache Kafka

*(Open Source Core)*

- ✓ Distributed event log & streaming
- ✓ Producers, Consumers, Topics
- ✓ KRaft / ZooKeeper consensus
- ✓ Kafka Streams (Java library)
- ✓ ksqlDB (community edition)
- ✓ Kafka Connect API
- ✓ Basic CLI tools

## Confluent Platform

*(Apache Kafka + Enterprise Tooling)*

**★ Everything in Apache Kafka, PLUS:**

- ✓ Confluent Control Center (GUI)
- ✓ Confluent for Kubernetes operator
- ✓ Self-Balancing Clusters
- ✓ Cluster Linking (multi-DC, hybrid cloud)
- ✓ Tiered Storage (S3/GCS/Azure)
- ✓ REST Proxy, JMS Client, MQTT Proxy
- ✓ Security Plugins & enterprise connectors

01

# Enterprise Features

Control Center, Kubernetes, Connectors, Security Plugins & more

# Confluent Control Center

*A GUI-based system for managing and monitoring your entire Kafka estate*

## What Control Center Does

Control Center is the operational command hub for Confluent Platform. It gives platform operators a single pane of glass to:

- Manage Kafka Connect — create, edit, and monitor connectors to external systems without writing code
- Monitor data streams end-to-end, from producer to consumer, with guaranteed delivery visibility
- Measure end-to-end message latency and track consumer lag
- Build entire production data pipelines with zero application code
- Define and receive latency + completeness alerts via email or REST

### Zero-Code Pipelines

Full drag-and-drop pipeline builder. Define sources, transforms, and sinks without any application code.

### End-to-End Monitoring

Visual message tracing from producer all the way to consumer. Every message delivery confirmed.

### Latency Alerts

Define alert thresholds on delivery latency and stream completeness. Notify via email or external alerting.

### Connect Management

Create, edit, pause, and restart Kafka Connect jobs from a UI. No command-line required for ops.

# Confluent for Kubernetes & Connectors

*Kubernetes-native deployment and a 500+ connector ecosystem*

## Confluent for Kubernetes

### Kubernetes Operator

Extends K8s orchestration with Kafka-specific lifecycle logic — rolling updates, config changes, and cluster scaling.

### Simplified Deployment

Deploy a complete Confluent Platform cluster with a single Kubernetes manifest. No bash scripts or Ansible needed.

### Automated Lifecycle

Automated broker replacements, certificate rotation, rolling restarts, and replica-aware pod scheduling.

### GitOps Ready

Declarative CRDs integrate with ArgoCD, Flux, and other GitOps tools for full infrastructure-as-code workflows.

## Confluent Connectors Hub

**500+** Pre-built connectors available on Confluent Hub — tested and supported with Confluent Platform

### Databases

PostgreSQL, MySQL, Oracle, MongoDB, Cassandra

### Cloud Storage

S3, GCS, Azure Blob, HDFS

### Search & Analytics

Elasticsearch, Splunk, BigQuery, Snowflake

### SaaS & Apps

Salesforce, ServiceNow, GitHub, HTTP/REST

# Security, JMS & MQTT — Bridging Legacy Systems

*Extend Kafka to legacy messaging, IoT devices, and multi-tenant security*

## Security Plugins

Adds multi-tenant security to Confluent tools. The REST Proxy plugin authenticates incoming requests and propagates the principal to Kafka — enabling full multi-tenant security without modifying Kafka brokers.

- REST Proxy integration
- Principal propagation
- Multi-tenancy
- OAuth/LDAP ready

## Confluent JMS Client

Implements the JMS 1.1 standard API backed by Kafka brokers. Legacy JMS applications can migrate to Kafka streaming without any application rewrite — just swap the broker endpoint.

- JMS 1.1 compliant
- Drop-in broker replacement
- Zero app rewrite
- Java legacy migration

## MQTT Proxy

Allows IoT devices and gateways to publish directly to Kafka topics using the MQTT protocol — eliminating the need for a separate MQTT broker in the architecture.

- Direct IoT to Kafka
- No MQTT broker needed
- MQTT 3.1 / 5.0
- Gateway support

02

# Balancing & Replication

Self-Balancing Clusters, Auto Data Balancer, Cluster Linking & Tiered Storage

# Self-Balancing Clusters & Auto Data Balancer

*Automated workload distribution — zero manual partition tuning*

## Self-Balancing Clusters

*The next iteration of Auto Data Balancer — continuous, automatic monitoring.*

**⚡ Auto-detects imbalances**
Continuously monitors partition distribution, leader count, and broker load.

**🔄 Automatic rebalancing**
Triggers partition reassignment automatically based on configured rules.

**➕ Broker add/decommission**
Adding or removing brokers triggers automatic data redistribution — no manual tuning.

**⚙️ Configurable trigger modes**
Choose 'Only when brokers change' or 'Anytime' for continuous auto-balancing.

**🛡️ Production-safe throttling**
Rebalance traffic is throttled to prevent impact on live production workloads.

## Auto Data Balancer

*The original balancer — useful for understanding the monitoring model.*

### What ADB Monitors:

**Number of brokers**
Detects when brokers are added or go offline

**Partition sizes**
Flags brokers with disproportionately large partitions

**Number of partitions**
Identifies broker overload from too many assignments

**Leader count**
Ensures leader elections are spread evenly

# Cluster Linking & Confluent Replicator

*Multi-datacenter, hybrid cloud, and active-active replication strategies*

## Cluster Linking

Directly connects clusters and mirrors topics over a link bridge with no consumer group offsets to manage.

### Zero-copy mirroring

Topics mirrored at the broker level — no intermediate consumers

### Multi-DC support

Connect on-prem ↔ cloud or region-to-region clusters

### Consumer transparency

Consumers read mirrored topics as if local — no offset translation needed

### Hybrid cloud

Seamlessly bridge private datacenter with AWS, Azure, GCP clusters

## Confluent Replicator

Maintains multiple Kafka clusters across datacenters. Replicates both data and topic configuration.

### Active-Active Geo

Users access their nearest datacenter — optimised latency and failover.

### Centralised Analytics

Aggregate data from multiple Kafka clusters into one analytics hub.

### Cloud Migration

Sync on-prem applications to cloud deployments during migration.

# Tiered Storage

*Decouple compute from storage — keep petabytes of data on cheap object storage*

## Kafka Brokers (Hot Data — recent)

Fast NVMe local storage · Recent messages · Low latency reads · Active consumers served here

## Object Storage (Warm/Cold Data — historical)

Amazon S3 · Google Cloud Storage · Azure Blob · Any S3-compatible store

### Up to 10x Cost Reduction

Object storage is 10-50x cheaper than SSD broker storage

### Independent Scaling

Scale compute (brokers) and storage independently

### Infinite Retention

Keep data as long as needed without adding brokers

### Transparent Reads

Consumers read old and new data with the same API

03

# Schema Registry

Centralised schema management and zero-downtime schema evolution

# Confluent Schema Registry

*The schema governance foundation for loosely coupled event-driven systems*

## The Problem Without Schema Registry

- ✕ Services agree on message format ad hoc — only implicitly defined in code
- ✕ Schema definitions are duplicated across every service that produces or consumes that type
- ✕ When formats change, it's impossible to know what will break downstream
- ✕ New teams joining a project have no standard way to discover message schemas
- ✕ Runtime failures occur silently when incompatible versions reach production

## How Schema Registry Solves It

- ✓ RESTful interface stores and retrieves Avro, JSON Schema, and Protobuf schemas
- ✓ Every schema gets an immutable integer ID — cached by producers/consumers
- ✓ Tracks ALL schema versions for every Kafka topic centrally
- ✓ Enforces user-defined compatibility rules (BACKWARD, FORWARD, FULL)
- ✓ Prevents incompatible schemas from ever reaching production
- ✓ Supports data governance with full schema lineage and audit trail

# Schema Compatibility Modes & Avro Integration

*Configure schema governance rules per-topic and integrate with your build pipeline*

## Compatibility Modes

### BACKWARD

New schema can read data written with the previous version. Safe for adding optional fields.

### FORWARD

Previous schema can read data written with the new version. Safe for removing optional fields.

### FULL

Both backward and forward compatible. Most restrictive — only additive, non-breaking changes.

### NONE

No compatibility check. Use only for development or throwaway topics.

## Schema-Driven Avro Workflow

1. **Author schema in .avsc**
   Describe your domain object in Avro's JSON IDL. Source-control the file like any code.
2. **Pull Request review**
   Schema changes go through the same PR/review process as application code. Teammates see diffs.
3. **Maven/Gradle codegen**
   The Avro plugin reads .avsc and generates type-safe Java POJOs — no manual serialization code.
4. **Registry validates at build**
   CI checks schema compatibility at build time — catch breaking changes before deployment.
5. **Runtime enforcement**
   Schema Registry checks every produce/consume against stored schema IDs — zero silent failures.

04

# REST Proxy & Clients

Language-agnostic Kafka access via HTTP and native client libraries

# Confluent REST Proxy

*Full Kafka functionality over HTTP — access from any language or platform*

### Produce Messages

POST to /topics/{name} to send messages to any Kafka topic. Supports raw bytes, JSON, and Avro formats.

### Consume Messages

Create consumer instances and read from topics individually or as part of a named consumer group.

### Consumer Groups

Full consumer group semantics over HTTP — subscribe, poll, commit offsets, and manage group membership.

### Cluster Metadata

Query topic list, partition assignments, broker config, and replica status via REST endpoints.

### Schema Registry Bridge

Automatically translates JSON ↔ Avro. REST Proxy registers and looks up schemas on your behalf.

### Java Client Quality

The REST Proxy is backed by the official Confluent Java clients — battle-tested reliability from any language.

# Confluent Client Libraries

*High-performance, officially maintained clients across every major language*

## C / C++

*librdkafka*

- 800K+ msg/sec producer throughput
- 3M+ msg/sec consumer throughput

## Python

*confluent-kafka-python*

- High-performance via librdkafka bindings
- Schema Registry integration

## Go

*confluent-kafka-go*

- Full-featured, idiomatic Go API
- Native Schema Registry support

## .NET

*Confluent.Kafka*

- NuGet package, production-ready
- Full Avro + Schema Registry support

05

# ksqlDB & Community

Streaming SQL, Community Connectors, Docker images & CLI tools

# ksqlDB — The Event Streaming Database

*SQL for continuous, real-time processing of Kafka event streams*

**ksqlDB is a scalable, elastic, fault-tolerant event streaming database.**

Write SQL — not Java or Python — and deploy a continuous stream processing job on Kafka in minutes.

## Streaming ETL

Power your data pipelines with ksqlDB transformations. Filter, reshape, and enrich data within the pipeline so it lands clean in downstream systems — no separate ETL tool required.

```
CREATE STREAM clean_orders AS
  SELECT order_id,
    UCASE(customer) AS customer,
    amount * 1.1 AS total_with_tax
  FROM orders
  WHERE status != 'CANCELLED'
  EMIT CHANGES;
```

## Materialised Views

Precompute query results as data arrives and store them for instant reads. ksqlDB uses Kafka for storage and its own compute for continuous materialisation — no separate database.

```
CREATE TABLE customer_totals AS
  SELECT customer_id,
    COUNT(*) AS order_count,
    SUM(amount) AS lifetime_value
  FROM orders
  GROUP BY customer_id
  EMIT CHANGES;
```

## Event-Driven Microservices

Stateful stream processing runs on a server cluster while your stateless microservice reads enriched events and acts on them. Decouples complex state from application logic.

```
CREATE STREAM alerts AS
  SELECT user_id, event_type,
    WINDOWSTART AS window_start
  FROM clicks
  WINDOW TUMBLING (SIZE 5 MINUTES)
  HAVING COUNT(*) > 100
  EMIT CHANGES;
```

# Docker Images & CLI Tools

*Container-ready deployment and a rich command-line toolset*

## Docker Images

| Available on Docker Hub | |
|---|---|
| cp-zookeeper | Community |
| cp-kafka | Community |
| cp-schema-registry | Community |
| cp-kafka-rest | Community |
| cp-kafka-connect | Community |
| cp-ksqldb-server | Community |
| cp-enterprise-kafka | Commercial ★ |
| cp-enterprise-control-center | Commercial ★ |

## CLI Tools

### confluent

The main Confluent CLI. Manage local dev environments, Confluent Cloud, and on-prem Platform.

```
$ confluent local services start
$ confluent iam user list
$ confluent kafka topic list
```

### kafka-topics

Create, list, describe, alter, and delete Kafka topics.

```
$ --create --topic my-topic
$ --list --bootstrap-server localhost:9092
```

### kafka-console-producer / consumer

Send and receive test messages from the command line — useful for debugging.

# ksqlDB Supported Operations

*A full streaming SQL feature set — from simple filters to windowed aggregations*

## Filtering

Real-time fraud / threshold detection

```
SELECT * FROM transactions
  WHERE amount > 10000
  EMIT CHANGES;
```

## Transformations

Reshape and reformat event fields

```
SELECT order_id,
  TIMESTAMPTOSTRING(rowtime,'yyyy-MM-
dd')
  AS order_date FROM orders
  EMIT CHANGES;
```

## Aggregations

Running totals, counts, averages

```
SELECT region,
  COUNT(*) AS events,
  AVG(latency) AS avg_ms
FROM telemetry
GROUP BY region EMIT CHANGES;
```

## Stream Joins

Enrich events with reference data

```
SELECT o.order_id,
  c.email
FROM orders o
JOIN customers c
  ON o.cust_id = c.id
EMIT CHANGES;
```

## Windowing

Hourly / daily time-bucketed metrics

```
SELECT product_id,
  SUM(qty) AS units_sold
FROM purchases
WINDOW TUMBLING (SIZE 1 HOUR)
GROUP BY product_id
EMIT CHANGES;
```

## Sessionization

Session window analysis per user

```
SELECT user_id,
  COUNT(*) AS actions,
  WINDOWSTART, WINDOWEND
FROM clicks
WINDOW SESSION (30 MINUTES)
GROUP BY user_id
EMIT CHANGES;
```

# Confluent Platform — Architecture at a Glance

*How all components fit together in a production deployment*

### Producers

- Apps / Microservices
- IoT via MQTT Proxy
- REST Proxy clients
- Legacy JMS apps

### Consumers

- Apps / Analytics
- ksqlDB
- Kafka Streams

## Apache Kafka

### Schema Registry

- Avro / JSON Schema / Protobuf
- Compatibility enforcement
- Schema versioning & audit

### Kafka Connect

- 500+ Connectors
- Source & Sink
- Scalable workers

### Tiered Storage

- S3 / GCS / Azure
- Cold data offload
- Cost-effective scale

**Confluent Control Center — Monitoring · Alerting · Connect Management · Schema Browser**

# Key Takeaways

**01**

### Control Center simplifies operations

A single GUI for monitoring, alerting, and pipeline management — production Kafka without writing operational scripts.

**02**

### Self-Balancing eliminates manual partition tuning

Add or remove brokers and Confluent automatically redistributes workload with production-safe throttling.

**03**

### Schema Registry is non-negotiable at scale

Centralised schema governance with BACKWARD/FORWARD/FULL compatibility prevents silent runtime failures across teams.

**04**

### REST Proxy & Clients make Kafka universal

Any language — C, Python, Go, .NET, or HTTP — can produce and consume Kafka events with full reliability guarantees.

**05**

### ksqlDB brings SQL to stream processing

Streaming ETL, materialised views, and event-driven microservices — all expressible in SQL without deploying separate frameworks.

**06**

### Confluent Platform = Kafka + enterprise acceleration

Every Confluent release ships the latest Kafka core plus the tools teams need to operate it confidently at scale.